

Global Digital Bank — Activities 11 to 14

Activities 1–10 built up core OOP mechanics — classes, encapsulation, validation, exceptions, inheritance, method overriding, and abstraction — using a single evolving `Account` hierarchy. Starting at **Activity 11**, the project shifts from "writing correct classes" to "**designing a system that can change safely**." This is the exact point where the **SOLID principles** are introduced, one responsibility at a time, driven by concrete business change requests rather than by principle names alone.

- Activities 1–10 → OOP fundamentals (no SOLID focus yet)
- Activity 11 → Interface Segregation + Dependency Inversion begin (via `IAccount` + `Factory`)
- Activity 12 → Validates the DIP/ISP shift with interface-only testing
- Activity 13.1/13.2 → Single Responsibility + Open/Closed (rules extracted, then made extensible)
- Activity 14 → Open/Closed taken further (external config — extend without recompiling)

Each activity below is applicable in Java, Python, or another OOP language, with the features expected to be built and the criteria for successful execution.

Activity 11 — `IAccount` Interface & Account Factory

Problem Statement

So far, every part of the codebase that needs to create or operate on an account has depended directly on concrete classes (`SavingsAccount` , `CurrentAccount` , etc.). The bank is about to onboard three new engineering squads who will each build features (loans, transfers, statements) on top of the account system. If they all depend on concrete classes, every new account type forces changes across the entire codebase, and no one can swap implementations for testing.

Business ask: *"Give every team a single, stable contract to build against — one that doesn't change even as we add new account types."*

SOLID Principles Introduced

- **Interface Segregation Principle (ISP):** `IAccount` defines only the operations a generic client actually needs (deposit, withdraw, balance, account type) — not internal account bookkeeping.
- **Dependency Inversion Principle (DIP):** Client code depends on the `IAccount` abstraction, not on concrete account classes. Concrete classes depend on that same abstraction instead of client code depending on them.

Features to Build

1. An `IAccount` interface/contract exposing the operations every account type must support: `deposit()` , `withdraw()` , `getBalance()` , `getAccountType()` .
2. A common base implementation (`AbstractAccount`) that implements the shared logic once (status, PIN, validation, exception handling carried over from Activities 3–10), leaving only type-specific rules to subclasses.
3. Four concrete account types implementing/extending the abstraction: `SavingsAccount` , `CurrentAccount` , `FixedDepositAccount` (new), and `SalaryAccount` (new).
4. An `AccountFactory` with a single creation method that takes an account-type identifier and construction parameters, and returns an `IAccount` — hiding the concrete class entirely from the caller.

Execution Criteria

- No code outside `AccountFactory` and the concrete classes themselves should ever reference a concrete account class by name.
- Adding `FixedDepositAccount` and `SalaryAccount` should not require modifying any pre-existing account class.
- Calling `AccountFactory.createAccount("SAVINGS", ...)` should return an object usable purely through `IAccount` methods.

Activity 12 — Testing Interface & Factory

Problem Statement

Activity 11 introduced the abstraction, but the team needs proof that it actually decouples client code from implementation details — not just that it compiles. QA wants a test suite that could run unmodified even if the underlying account classes were completely rewritten tomorrow, as long as the

factory and interface contracts stay the same.

Business ask: *"Prove that your test code never has to know which class it's really talking to."*

SOLID Principles Reinforced

- **DIP (validated):** the test driver exercises accounts purely through `IAccount` references obtained from `AccountFactory`.
- **ISP (validated):** every test call maps to a method that a generic client genuinely needs — no test should require casting down to a concrete class to work.

Features to Build

1. A test/driver module that creates one instance of each of the four account types **exclusively via** `AccountFactory`.
2. Assertions/print statements that verify deposit, withdraw, and balance behavior using only `IAccount`-typed references (no downcasting to `SavingsAccount`, `CurrentAccount`, etc.).
3. At least one negative-path test (e.g., invalid account type passed to the factory) confirming the factory fails predictably rather than silently.

Execution Criteria

- The test file should compile/run without importing any concrete account class directly — only `IAccount` and `AccountFactory`.
- Running the suite should print or assert correct type-specific behavior (e.g., different minimum balances) even though the test code itself never mentions the concrete type.

Activity 13.1 — Rules Engine (Conditional Logic)

Problem Statement

Business rules — minimum balances, interest rates, overdraft limits — are currently scattered as hardcoded values inside each account class. Every time the bank's product team changes a policy (e.g., raises the Savings minimum balance), a developer has to hunt through multiple account classes

to find and update the number, risking inconsistency.

Business ask: *"Give us one place where all account policy numbers live, so nobody has to touch account classes just to change a number."*

SOLID Principle Introduced

- **Single Responsibility Principle (SRP):** account classes should be responsible for *behavior* (deposit/withdraw mechanics), not for *knowing business policy values*. A new `AccountRulesEngine` becomes solely responsible for answering "what is the rule for this account type?"

Features to Build

1. An `AccountRulesEngine` (implemented as a singleton, since policy data should be centralized and consistently loaded) with methods such as:
 - `getMinimumBalance(accountType)`
 - `getInterestRate(accountType)`
 - `getOverdraftLimit(accountType)`
2. Conditional (if/else) logic inside the engine mapping account type strings to their correct policy values, based on realistic bank figures.
3. A `validateWithdrawal(...)` style method combining minimum balance and overdraft rules into a single reusable check.
4. Account classes updated to delegate policy lookups to the rules engine instead of hardcoding values internally.

Execution Criteria

- Changing a policy number should require editing only the rules engine — zero changes to any account class.
- All four account types should correctly resolve their respective minimum balance, interest rate, and overdraft rules through the engine.

Activity 13.2 — Rules Engine (Map-Based Refactor)

Problem Statement

The if/else-based rules engine from 13.1 works, but the product team has announced that tenure-based pricing is coming — different rates depending on how long a customer has held the account. An if/else chain that already checks account type would now also need to branch on tenure, doubling in complexity and becoming slow and fragile to extend.

Business ask: *"We're going to add new pricing tiers every quarter. Don't make us rewrite branching logic each time."*

SOLID Principle Reinforced

- **Open/Closed Principle (OCP):** the rules engine should be **open for extension** (new account types, new tenure tiers) but **closed for modification** of its core lookup mechanism. Adding a new rule should mean adding a new data entry, not editing conditional logic.

Features to Build

1. Refactor `AccountRulesEngine` internals to use a nested lookup structure (e.g., a map/dictionary keyed by account type, and within that, keyed by tenure bucket) instead of if/else branching.
2. A small internal `Rule` data holder (interest rate, minimum balance, feature name) stored as the value in the lookup structure.
3. A `loadRules()` -style method that populates the lookup structure once at startup.
4. All existing public methods (`getMinimumBalance` , `getInterestRate` , etc.) preserved with the same signatures, so no caller code changes — only the internal implementation changes.

Execution Criteria

- Adding a new tenure bucket or account type should be achievable by adding one new entry to the lookup structure — no new `if / else` branches.
- Lookup performance should be effectively constant-time ($O(1)$) rather than proportional to the number of rules ($O(n)$).
- All Activity 13.1 test cases should still pass unchanged against the refactored engine.

Activity 14 — Externalized Rules via Configuration Files

Problem Statement

Even with the map-based engine, every policy change still requires a code change and a redeploy — unacceptable for a bank that adjusts interest rates and limits regularly, sometimes on short notice from regulators or leadership. The operations team wants to change a number in a config file and have it take effect without recompiling or restarting the whole application.

Business ask: *"Let compliance and product teams change policy numbers without needing a developer to redeploy the app."*

SOLID Principle Extended

- **Open/Closed Principle (OCP), taken to its logical conclusion:** the system is now closed even to *code* modification for routine policy changes — extension happens purely through external configuration, the most complete form of "open for extension, closed for modification."

Features to Build

1. One configuration file per account type (e.g., `savings.properties` / `savings.config` , `current.properties` , `fixeddeposit.properties` , `salary.properties`) storing key-value pairs for minimum balance, interest rate, overdraft limit, and tenure-based variations.
2. An `AccountRulesPropertiesLoader` (or equivalent) responsible only for reading these files into an in-memory structure at startup.
3. Integration of the loader with `AccountRulesEngine` , so the engine's lookup methods are now backed by externally loaded data rather than hardcoded maps.
4. A `reloadRules()` capability that re-reads the configuration files on demand, without restarting the application ("hot reload").

Execution Criteria

- Changing a value in a properties file and calling `reloadRules()` should change the engine's returned values on the next lookup — with no code change or recompilation.
- The rules engine's public interface (from 13.1/13.2) should remain unchanged; only where the data comes from has changed.
- Missing or malformed configuration entries should fail predictably (e.g., a sensible default or a clear error) rather than silently corrupting account behavior.